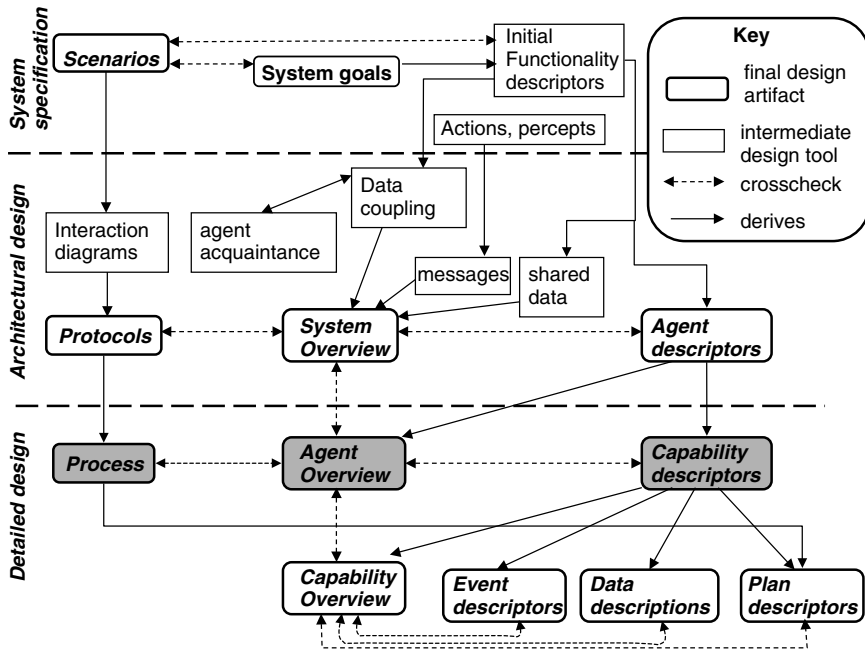


Detailed Design: Agents, Capabilities and Processes



Phases, artifacts and relationships in the design process

During architectural design, we developed an overview of which agents are in the system (the system overview diagram) and how these will interact to achieve system goals (the protocols), as well as descriptors for each agent. In the detailed design, we take these and, for each individual agent, flesh out what capabilities are needed for the agent to

fulfil its responsibilities as outlined in the functionalities it contains. We also develop the protocol specifications to indicate more of the internal processing of individual agents.

As we get into greater detail (in Chapter 9), we further develop the capability descriptions to specify the individual plans, beliefs and events needed within the capabilities. We also further develop the views that show processing of particular tasks within individual agents. It is during this final phase of detailed design that the methodology becomes specific to agents that use event-triggered plans in order to achieve their tasks. This will be discussed further in the following chapter.

In this initial part of detailed design, the aspects that are addressed are as follows:

1. Decisions regarding the capabilities needed by an agent to fulfil its tasks.
2. Description of the relationships between the capabilities using an *agent overview diagram*.
3. Development of the protocol specifications to indicate some of the internal processing of individual agents.
4. Summarizing information about each capability in capability descriptors.

For the purpose of continuing our running example, we will focus on the Stock Manager agent type.

As with all software engineering, it is appropriate, especially at the detailed design stage, to combine top-down design with bottom-up. In the framework of agent design, top-down design translates to designing the capabilities and their interactions, and then the plans that will be a part of those capabilities. The alternative practice of bottom-up design involves directly designing plans that will accomplish the tasks of the agent, later grouping these into the relevant capabilities. As part of the bottom-up design process, there is also likely to be implementation intermixed with the design. Use of a design tool that generates skeleton code greatly facilitates this process. The reality is that there is usually a mix of bottom-up and top-down development. The larger and more complex the agent is, the greater focus there is likely to be on top-down design, first specifying the capabilities.

In the presentation, we will present first the design of capabilities, then of plans and events. Whichever process is used in the actual development, it is useful in the design documentation to present it from a top-down perspective, to allow the reader to gradually understand greater detail.

8.1 CAPABILITIES

In the architectural design, we have determined what functionalities should be encompassed by each agent, and the list of these functionalities is a natural starting place for the agent capabilities.

Sometimes, there is also functionality akin to ‘library routines’ that is required in multiple places – either within multiple agents, or within several capabilities within a single agent. Such functionality should also be extracted into a capability that can then be included into other capabilities or agents as required.

Capabilities may be nested within other capabilities and thus this model allows for arbitrarily many layers within the detailed design, in order to achieve an understandable complexity at each level. As the design develops, what was originally a single functionality may well be split into smaller modules, each of which is a capability. It is also sometimes the case that a capability is formed by *merging* two or more functionalities where the functionalities in question are closely related and would result in very simple capabilities if left apart. As previously, the principle of *cohesion* is paramount. Each capability should be a well-defined collection of plans, using particular beliefs or data, which addresses a specific set of goals (or sub-goals) of the agent.

For each capability, we need to determine the goals it is to achieve within the system. If it is derived from a functionality, this information already exists, although there may be some rearrangements at this stage. It is useful, as the design is refined, to also modify the goal and functionality information, if this makes sense.

 **TIP: When to modify goals and functionalities developed in System Specification:** This is of course a matter of judgement. If the refinements involve things like splitting a functionality into two capabilities, or adding a new low-level capability that will be included in other capabilities, then there is no need at all to consider modifying functionalities and goals. Indeed, it can be counter-productive to do so, as it can clutter the high-level design with unnecessary detail, making it harder to understand. On the other hand, if the change is significant and reflects rethinking the design, then it can be useful to also consider updating the functionalities. For example, if a goal G is assigned to functionality F but during the design process it is determined that it is better to achieve the goal in capability C (which corresponds to functionality F_2), then it can be useful to update the functionalities by moving G from F to F_2 . The main criterion is understandability. If it makes the high level more understandable, then we would move it. If not, then we would leave it, but note at the detailed design the reason for the move.

Electronic Bookstore: Case study

Taking the example of the Stock Manager agent, we develop the capabilities on the basis of the included functionalities. The capabilities and their related goals are as follows:

- **Stock managing** (Goals: Have books in stock, Log books arriving, Log books outgoing, Reorder stock)
- **Pricing** (Goals: Competitive prices, Set prices competitively)
- **Cataloging** (Goals: Update catalogue)
- **Managing competition** (Goals: Competitive prices, Lower book price, Monitor competitive response, Restore book price).

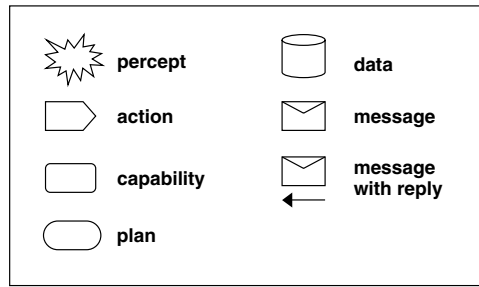


Figure 8.1 Graphical symbols used in the agent overview diagram

8.2 AGENT OVERVIEW DIAGRAMS

The information as to what goal(s) each capability is to achieve, combined with the information of the protocols the agent engages in (from the previous phase), is usually sufficient to enable us to draw the *agent overview diagram*. This diagram shows the relationships between the capabilities, thus providing a top level view of the agent internals. It is very similar in style to the system overview diagram, but instead of agents within a system, it shows capabilities within an agent. This diagram shows the top-level capabilities of the agent and the message flow between these capabilities, as well as data internal to the agent. It is also possible to have some plans at this level if there is some agent functionality that is sufficiently simple that it would be redundant to place it within a capability. Figure 8.1 shows the graphical symbols for the Agent Overview diagram.

To develop the Agent overview diagram, one begins with the interface to the agent as specified in the System Overview diagram. The capabilities are then added to the diagram, and each interface element is linked to at least one capability. The protocol descriptions can then be used to guide the specification of the messages that need to pass between capabilities. Development of initial activity diagrams (based on UML, but described further below) can also assist in this process.

TIP: It is often useful to link the names of messages to the name of the sub-goal to which they are related. For instance, when developing protocols, goals in scenarios can guide the naming of messages in the protocols. Similarly, there are typically a number of sub-goals to be achieved, by a particular capability, and it can be useful for the message indicating the need for achievement of the sub-goal to have a name related to that sub-goal. For example, one of the sub-goals of the *Profile Monitor* functionality, within the *Customer Relations* agent, is to *Update customer profile*. When this is needed, the relevant capabilities (*Purchasing*, *Delivery Manager*, *Online Interaction*) may well send an *Update customer profile message*. An alternate name for this message could also have been *Request update customer profile*. This helps in linking messages to the goals to be achieved, as specified in the initial phases.

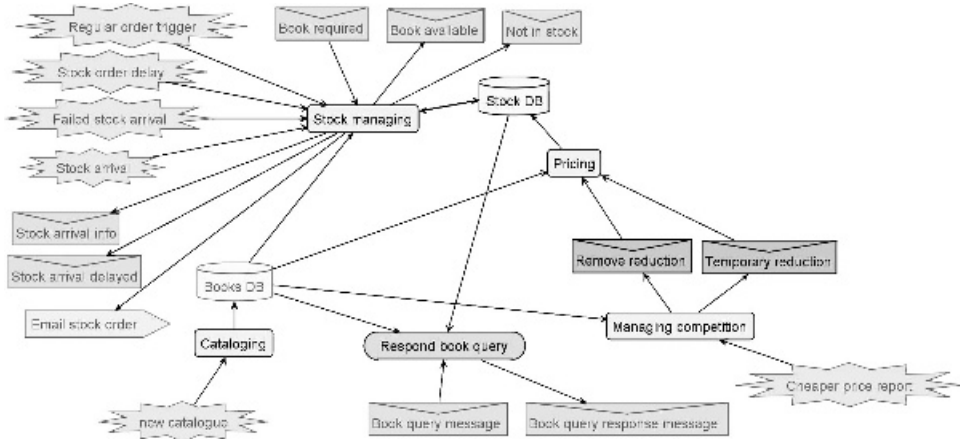


Figure 8.2 Agent overview diagram: stock manager

Electronic Bookstore: Case study

Figure 8.2 shows an agent overview diagram for a Stock Manager agent in the electronic bookstore. Faded entities are the interface to the agent, that is, they depict external entities such as incoming events, actions and percepts.

This agent has the capabilities of Stock managing, Pricing, Cataloging and Managing competition. Note that there is also a single plan for responding to queries on the Books DB and the Stock DB.

8.3 PROCESS SPECIFICATIONS

During system specification, high-level processes were specified by scenarios, which were then refined and more fully specified using interaction diagrams and protocols during architectural design. In detailed design, we also want some mechanism to specify process as well as structure. For this, we use a slight variant of UML activity diagrams. Figure 8.3 illustrates the concepts and the notation of our extended variant of activity diagrams. Note that rather than using swimlanes to separate activities of different agents, as would perhaps be the most obvious modification of UML, we choose instead to focus only on the activity within a single agent, indicating interaction with other agents via the inclusion of messages within the diagram. We believe this is a better choice, partly because it avoids diagrams being overly cluttered, and perhaps more importantly, because it allows for modular development of agents, with shared knowledge only about the interface. It may also be useful in some circumstances to give a process specification diagram which does put together the specifications from all agents involved in that process.

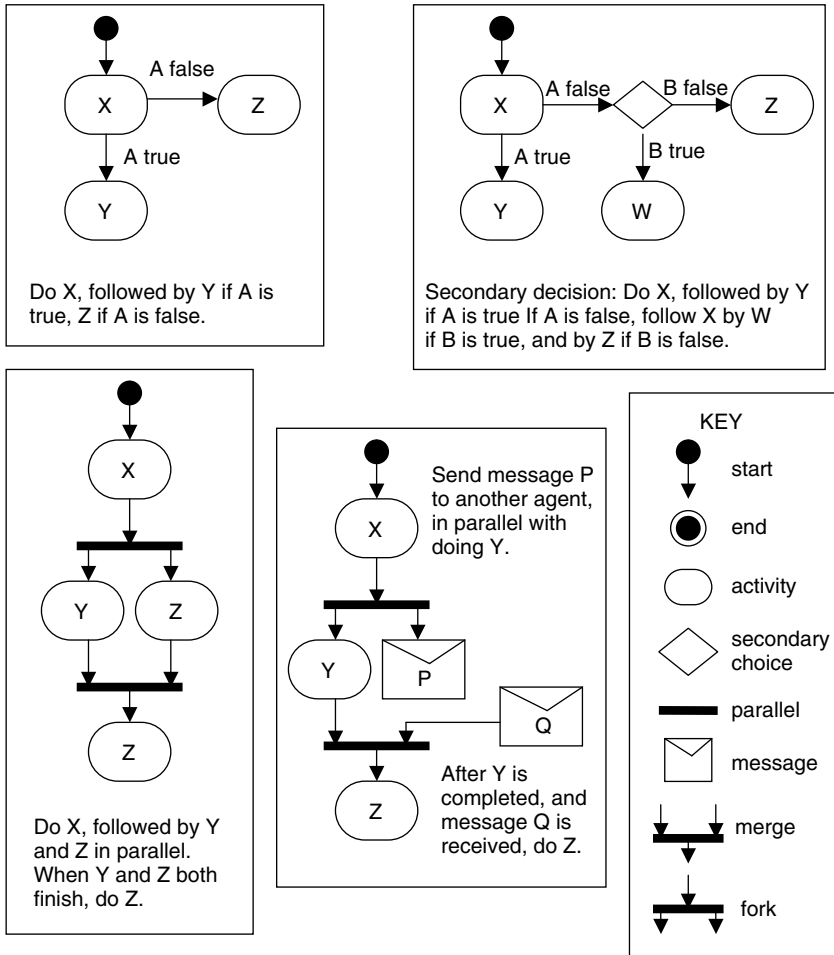


Figure 8.3 Diagram illustrating notation for process specifications

For further information on standard Activity Diagrams, see (Fowler and Kendall 2003).

We identify the process diagrams to be developed by looking at the protocols involving the Stock Manager, as well as the scenarios developed and the goals of the agent. Some process diagrams will not be identified from the protocols as they do not involve significant interaction with another agent.

Electronic Bookstore: Case study

By looking at the scenarios or protocols originating with the Stock Manager agent, and by looking at the goals of the agent, we come up with 5 process diagrams for the Stock Manager agent. They are

- Stock maintenance
- Stock arrival
- Stock delay
- Cataloging
- Pricing reduction.

Stock maintenance and Stock arrival activity diagrams are shown in Figure 8.4.

Notice that stock maintenance is not derived from a protocol. However, the existence of such goals as *Reorder stock* and *Have books in stock* indicate the need for a specification of a process for achieving this. As can be seen, there is no need for a protocol as there is only a single message involved at the end of (one branch of) the activity.

An outgoing message in a process diagram should be able to be mapped either to a process diagram within the receiving agent, or to a possible entry point within a process diagram within the receiving agent. The former is of course simply a special case of the latter, where there is only one entry point and it is the relevant message. Figure 8.5 shows the process for *Updating a user profile* within the Sales Assistant Agent, and the related activity of *Manage customer profile* within the Customer Relations Agent. The *Update customer profile* message from the Sales Assistant activity diagram is one of the possible entry points into the *Manage customer profile* activity diagram in the Customer Relations agent.

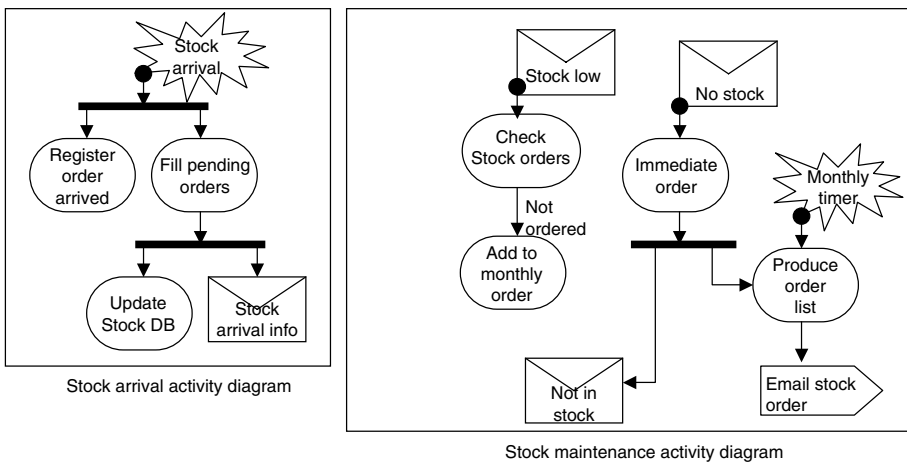


Figure 8.4 Diagram of some of the processes in the stock manager agent

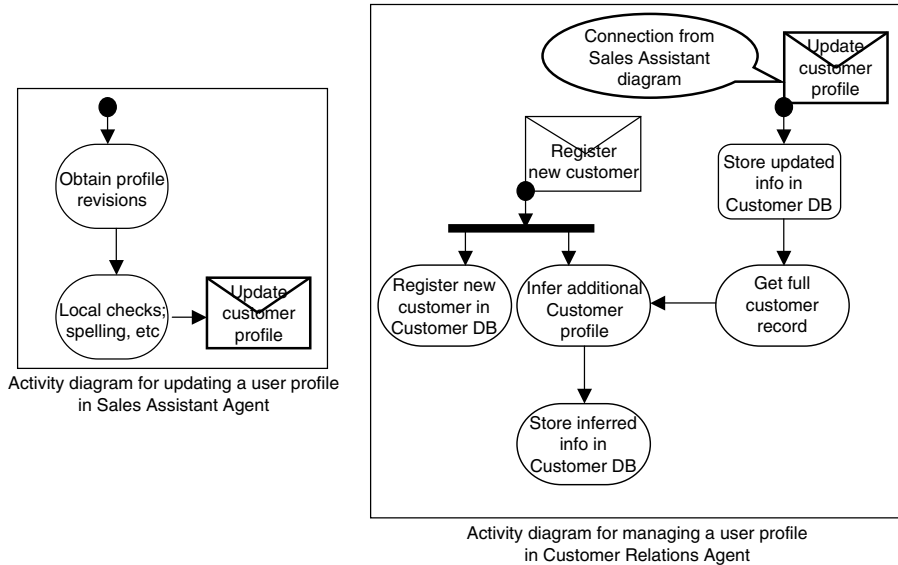


Figure 8.5 Diagram of manage customer profile process in the customer relations agent

8.4 DEVELOP CAPABILITY AND PROCESS DESCRIPTORS

We continue the practice of having structured textual descriptors for each of the significant entities in the system. At this stage, we develop descriptors for each of the Capabilities, and for each of the Process Diagrams. The information for the Capability Descriptor can be extracted almost entirely from existing information, but it is useful to have it gathered in the one place. Use of a support tool can automate most of the process of developing the capability descriptors.

Each Capability Descriptor should have the following fields:

- **Name:**
- **Description:** Brief natural language description.
- **Goals:** Can often be taken from a related functionality, but may need refinement.
- **Processes:** A list of the processes within the agent that the capability is involved in.
- **Protocols:** A list of the system-level protocols that this capability is involved in.
- **Outgoing messages:** A list of messages sent, along with which capability/agent they are sent to, and which goal they relate to.
- **Incoming messages:** A list of messages received, along with which capability/agent they are received from and which goal they relate to.

- **Internal messages:** A list of messages internal to the capability.
- **Percepts:** A list of percepts received directly by this capability.
- **Actions:** A list of actions done directly by this capability.
- **Included capabilities:** Any included capabilities, along with the goals associated with those capabilities.
- **Data used: Imported:** Data imported to the capability
- **Data produced: Exported:** Data exported from the capability.
- **Data internal:** Data used internally.
- **Included capabilities:** Capabilities within this capability.
- **Included plans:** Plans within this capability.
- **Notes:** Any relevant design notes.

Electronic Bookstore: Case study

Here is the capability descriptor for the Ordering capability within the Stock Manager Agent.

Name:

Description: This capability manages the ordering of stock from suppliers – either on a regular basis, or if necessary when stock runs out

Goals: Order stock

Processes: Stock maintenance

Protocols: Book ordering protocol

Outgoing messages: Book available, not in stock

Incoming messages: Book required

Internal messages: Modify monthly order, decide supplier, Get number required, No stock, stock low

Percepts: Regular order trigger

Actions: E-mail stock order

Included capabilities: None

Data used: Imported Pending Orders, Books DB

Data produced: Exported Stock Orders, Pending Orders

Data internal: Monthly Order

Included capabilities: none

Included plans: Check stock, Add to order, Out of stock response, Number by index, Get number by price, Get number by sales, Decide supplier by time, Decide supplier by price, Delete items, Add to supplier order, Build monthly orders

Notes: (none).

The Process Descriptor also contains information that is available in the rest of the system, but can be useful to have accumulated in one place, so that it can be viewed along with the Process Diagram to obtain additional information such as the receiving agents of a message.

We use a Process Descriptor with the following fields:

- **Name:**
- **Description:**
- **Activities:** May be existing (sub-)goals, or newly identified such
- **Triggers:** List of goals/messages/percepts
- **Messages:** <message name, to-agent>
- **Protocols:** Those protocols this process participates in
- **Notes:** (none).

Electronic Bookstore: Case study

Here is the process descriptor for the 'Stock maintenance' process, within the Stock Manager Agent.

- **Name:** Stock maintenance.
- **Description:** The activity whereby there is an attempt to maintain sufficient stock to immediately fill orders. The activity is responsive to immediate demands as well as maintaining stock levels from month to month.
- **Triggers:** Book required, Monthly timer.
- **Activities:** Check Stock DB, Immediate order, Add to monthly order, Produce order list.
- **Messages:** <Update customer profile, to Customer Relations Agent>, <Book purchase, to Delivery Manager Agent>
- **Protocols:** Book ordering protocol, Stock arrival protocol, Query late books protocol.
- **Capabilities:** Stock managing.
- **Notes:** (none).